DMA on the STM32F767ZI

Reference notes for spaced repetition: architecture, streams and channels, arbiter, transfer types, the enable sequence, and how this project sends a USART3 message by DMA.

Sources: ST RM0410 Rev 5 (STM32F76xxx/77xxx reference manual) and ST AN4031 Rev 3 (DocID022648, Using the STM32F2, STM32F4 and STM32F7 Series DMA controller); full list in section 13. Figures and tables marked *RM0410* are reproduced from the reference manual for personal study. Project files: `app/src/main.c`, `drivers/src/stm32f7xx_it.c`. Draft, 2026-10-29.

#	Section
1	What DMA is, why and when to use it
2	Where the DMA sits in the system
3	Inside one DMA controller
4	Streams and channels
5	Arbiter
6	Types of transfer (direction, modes, widths, FIFO, burst)
7	Interrupts, flags and errors
8	Enabling sequence
9	How this project uses DMA
10	Things to watch
11	Register quick reference
12	Flashcards
13	References

1. What DMA is, why and when to use it

1.1 What DMA is

Direct memory access (DMA) is a hardware block that copies data between peripherals and memory, or between two memory areas, without the CPU executing a load and a store for every item. RM0410 (8.1) puts it this way: DMA provides "high-speed data transfer between peripherals and memory and between memory and memory", and data "can be quickly moved by DMA without any CPU action. This keeps CPU resources free for other operations."

Physically, the STM32 DMA is a bus master of its own. AN4031 (section 1) describes it as an AHB module with three ports: a **slave port** the CPU uses to program it, and two **master ports** (peripheral and memory) that let the DMA start transfers itself. The CPU only writes the job description (source, destination, count, options) into the DMA registers. From then on the DMA does the copying, triggered by the peripheral's request signal, and signals the CPU with an interrupt when the block is finished.

1.2 Without DMA vs with DMA

	CPU-driven (polling or interrupt per item)	DMA-driven
Who moves each item	CPU: wait for the flag, read the source, write the destination	DMA hardware, on each peripheral request
CPU involvement	Every item (or one interrupt per item)	Once to set up, once at the end (TC interrupt); optionally at half transfer
CPU during the transfer	Busy, or constantly interrupted	Free to run other code, or sleep
Timing	Depends on CPU load and interrupt latency	Served in hardware within a few bus cycles
Code	Simple loop	More setup, flags, interrupts; cache care on F7

Worked example from this project. `usart3_send()` sends the 13-byte message by polling. At 115200 baud, 8N1, each character is 10 bits, so the message takes 13 x 10 / 115200 = **about 1.13 ms**. At the 16 MHz HSI clock this project runs on, that is **about 18,000 CPU cycles** spent waiting on TXE. With DMA, the CPU writes a handful of registers, sets DMAT, and is free for the whole 1.13 ms; it is interrupted only at half and full transfer.

1.3 Benefits (from ST's documentation)

Benefit	What it means	Source
Offloads the CPU	Transfers "take place in the background, without the intervention of the Cortex-Mx processor". The CPU "is only interrupted when a whole data block is available for processing".	AN4031 1
Large data, little impact	"Large amounts of data can be transferred with no major impact on the system performance."	AN4031 1
Cheaper and lower power than per-peripheral buffers	A central buffer in SRAM fed by DMA is "less expensive in terms of silicon and power consumption" than each peripheral having its own local storage.	AN4031 1
Low latency, parallel paths	Two AHB master ports work at the same time, and DMA1/DMA2 have direct paths to the APB bridges, so DMA serving slow APB peripherals does not stall the CPU.	AN4031 2.2.1, 5
Width conversion and bursts	The FIFO lets source and destination widths differ (packing/unpacking) and speeds up transfers with incremental bursts.	AN4031 5; RM0410 8.2
Flexible routing	16 streams in total, each selecting one of up to 16 peripheral requests.	RM0410 8.1; AN4031 5
Lower power	In Sleep mode the Cortex-M7 is stopped but peripherals keep running, and DMA1/DMA2 can stay clocked (DMA1LPEN/DMA2LPEN). The CPU can sleep (WFI) while DMA moves data and be woken by the DMA interrupt.	RM0410 4.3, 5.3.15

1.4 Typical use cases

Each of these appears in the F767 request mapping tables (RM0410 Tables 27 and 28) or as a worked example in AN4031.

Use case	Direction / mode	Example on the F767
UART transmit of a buffer or log message	M2P, normal	USART3_TX on DMA1 S3 C4 (this project)
UART receive of unknown-length data	P2M, circular ring buffer	USART3_RX on DMA1 S1 C4
ADC continuous / scan sampling	P2M, circular or double buffer	ADC1 on DMA2 S0/S4 C0 (AN4031 3.2.1 example: ADC to SRAM)
SPI full-duplex streaming (displays, flash, sensors)	Two streams: P2M for RX, M2P for TX	SPI1 on DMA2 (AN4031 3.2.2 example)
Audio in/out	Circular / double buffer	SAI1, SAI2, SPDIFRX
Waveform generation	M2P, circular, timer-paced	DAC1/DAC2 on DMA1 S5/S6 C7

Use case	Direction / mode	Example on the F767
Timer-driven updates (PWM patterns, LED strips)	M2P on a timer update/CC request	TIMx_UP, TIMx_CHy requests
Storage and external memory	P2M / M2P, burst	SDMMC1/2, QUADSPI (AN4031 4.9.2)
Memory copy or fill	M2M (DMA2 only)	Software-triggered by setting EN

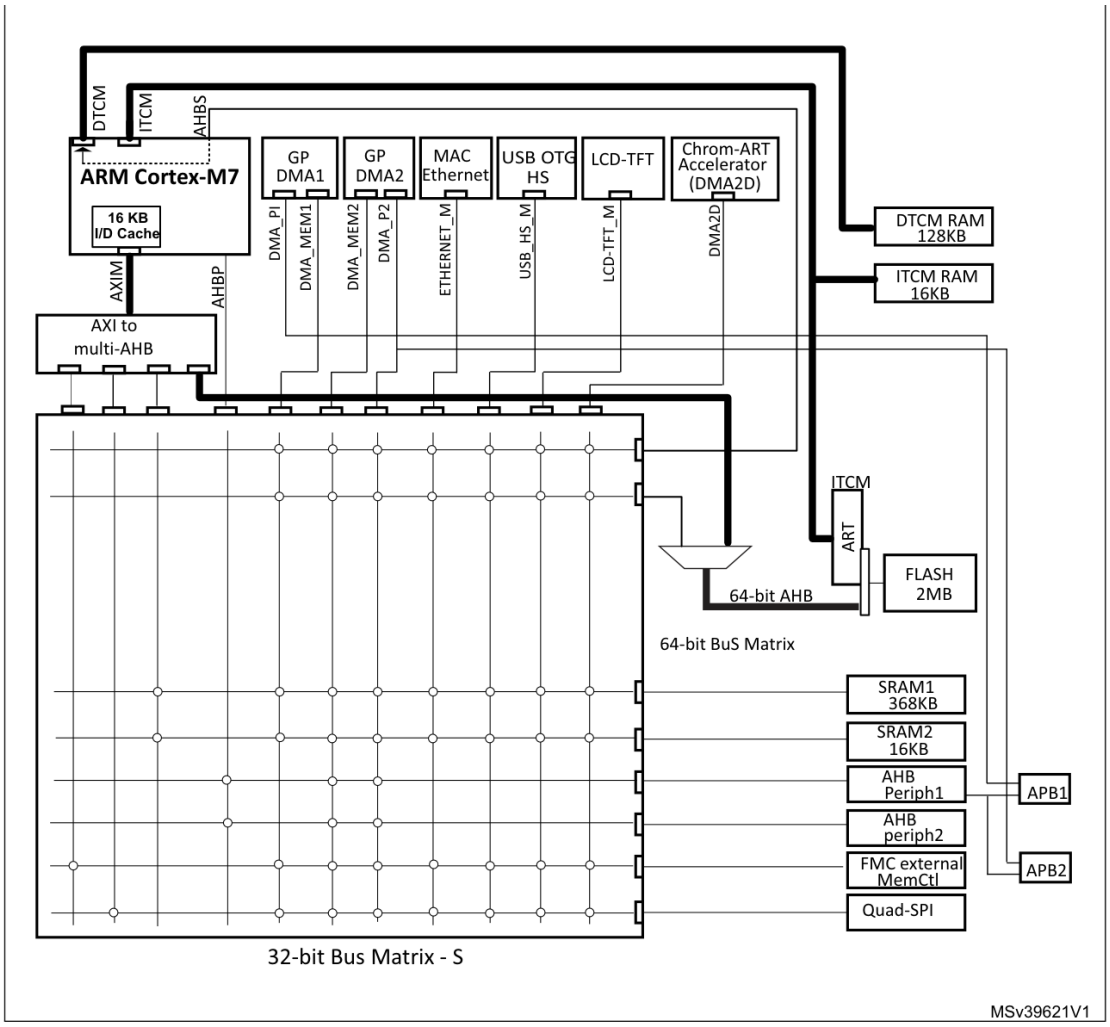
1.5 When DMA is not worth it, and its costs

- **Very short or rare transfers.** For a few bytes, the setup and interrupt handling can cost more than a simple polling loop.
- **Bus sharing.** DMA competes with the CPU and other masters on the bus matrix. AN4031 (2.1.3) shows a DMA access delayed by 8 AHB cycles when the CPU wins arbitration on an interrupt entry, and recommends avoiding contention on the same memory.
- **Cache coherency on the F7.** With cacheable buffers the software must clean the D-cache before a DMA read of memory and invalidate it before reading DMA-written memory, or use an MPU-configured non-cacheable region (AN4031 4.10).
- **Limited and fixed routing.** Each request is wired to specific stream/channel pairs, so two peripherals can compete for the same stream.
- **More to get right.** Flags, enable order, and error handling (sections 7, 8 and 10).

Remember: DMA = a second bus master that copies data on peripheral requests while the CPU does other work. Use it for blocks or continuous streams; poll for a few bytes.

2. Where the DMA sits in the system

The F767 has two general-purpose DMA controllers, **DMA1** and **DMA2**. Each one is a bus master with two AHB master ports: a **memory port** and a **peripheral port**. In the system architecture they appear as GP DMA1 and GP DMA2 at the top of the bus matrix.

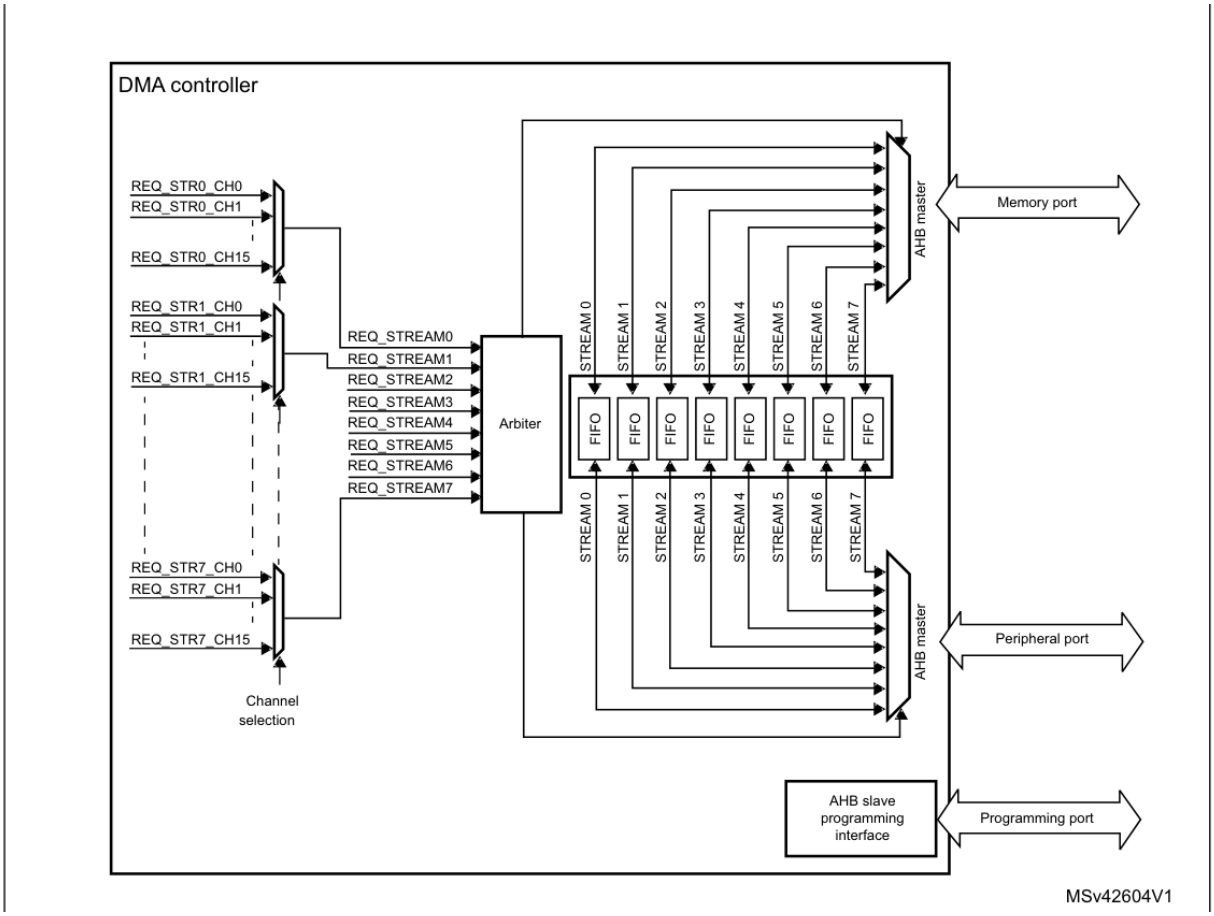


RM0410 Figure 1. System architecture for STM32F76xxx/77xxx (p. 72). Note DMA_PI (DMA1 peripheral port) bypasses the bus matrix and goes straight to APB1.

- **DMA1's peripheral port (DMA_PI) is not on the bus matrix.** It connects only to the APB1 peripherals. That is why **only DMA2 can do memory-to-memory** transfers.
- USART3 is on APB1, so its DMA requests are served by DMA1.
- The DMA memory ports (DMA_MEM1, DMA_MEM2) reach Flash, SRAM1/2, FMC and Quad-SPI through the bus matrix, so a `const` string in Flash is a valid DMA source.
- The Cortex-M7 L1 D-cache sits between the CPU and the bus matrix; the DMA does not see it (see section 10).

Remember: DMA1 = APB1 peripherals only on its peripheral port, so no memory-to-memory. DMA2 can do all three directions.

3. Inside one DMA controller



RM0410 Figure 23. DMA block diagram (p. 250).

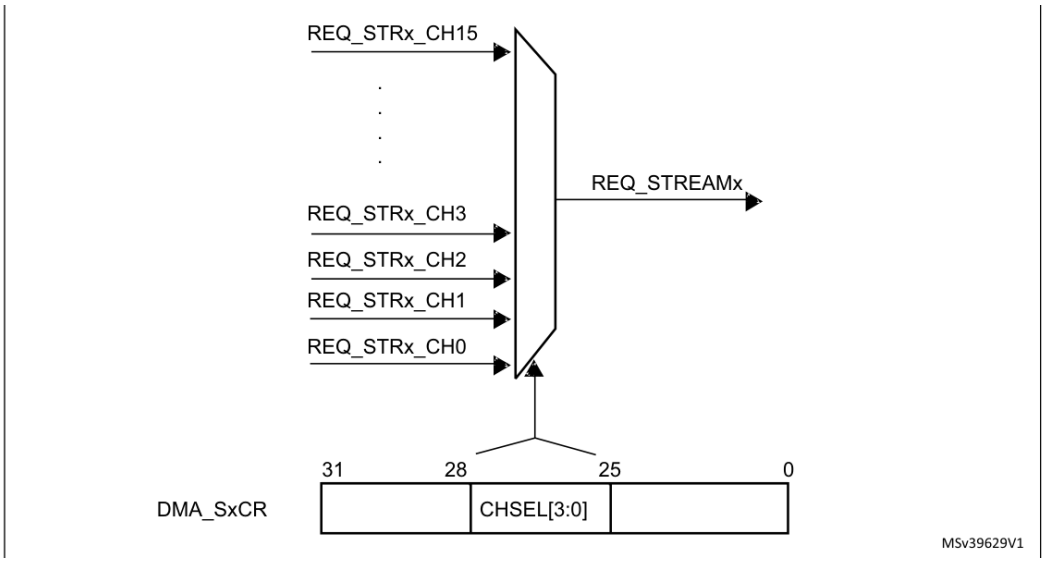
Item	Per controller
Streams	8 (Stream 0 to 7). Each stream is an independent, one-direction transfer engine.
Channels per stream	Up to 16 request inputs, selected by CHSEL[3:0] (bits 28:25).
FIFO	One 4-word (16-byte) FIFO per stream.
Items per transfer	1 to 65535 (NDTR is 16 bits), counted in PSIZE units.
Arbiter	One per controller, picks which stream gets each AHB port next.
Programming port	AHB slave interface. Supports 32-bit accesses only.
Event flags	5 per stream (HT, TC, TE, FE, DME), ORed into one interrupt per stream.

Data path for one transfer: the peripheral raises its request line, the stream's channel multiplexer passes it to the arbiter, the arbiter grants the stream, and the stream reads from the source port into its FIFO and writes from the FIFO to the destination port. NDTR is decremented after each item.

Handshake (RM0410 8.3.3): the peripheral asserts a request, the DMA accesses the peripheral and sends an acknowledge, the peripheral releases the request, then the DMA releases the acknowledge. Only then can the peripheral request the next item.

4. Streams and channels

A **stream** is the hardware that moves the data. A **channel** is a request line from a peripheral. Each stream has a 16-input multiplexer, and CHSEL picks which channel (peripheral) is allowed to trigger it. Only one channel is active on a stream at a time.



RM0410 Figure 24. Channel selection (p. 252).

The fixed wiring of peripherals to stream/channel pairs is given in the request mapping tables. You do not choose freely: you pick a stream/channel pair from the table that has your peripheral.

Peripheral requests	Stream 0	Stream 1	Stream 2	Stream 3	Stream 4	Stream 5	Stream 6	Stream 7
Channel 0	SPI3_RX	SPDIFRX_DT	SPI3_RX	SPI2_RX	SPI2_TX	SPI3_TX	SPDIFRX_CS	SPI3_TX
Channel 1	I2C1_RX	I2C3_RX	TIM7_UP	-	TIM7_UP	I2C1_RX	I2C1_TX	I2C1_TX
Channel 2	TIM4_CH1	-	I2C4 _RX	TIM4_CH2	-	I2C4 _TX	TIM4_UP	TIM4_CH3
Channel 3	-	TIM2_UP TIM2_CH3	I2C3_RX	-	I2C3_TX	TIM2_CH1	TIM2_CH2 TIM2_CH4	TIM2_UP TIM2_CH4
Channel 4	UART5_RX	USART3_RX	UART4_RX	USART3_TX	UART4_TX	USART2_RX	USART2_TX	UART5_TX
Channel 5	UART8_TX	UART7_TX	TIM3_CH4 TIM3_UP	UART7_RX	TIM3_CH1 TIM3_TRIG	TIM3_CH2	UART8_RX	TIM3_CH3
Channel 6	TIM5_CH3 TIM5_UP	TIM5_CH4 TIM5_TRIG	TIM5_CH1	TIM5_CH4 TIM5_TRIG	TIM5_CH2	-	TIM5_UP	-
Channel 7	-	TIM6_UP	I2C2_RX	I2C2_RX	USART3_TX	DAC1	DAC2	I2C2_TX
Channel 8	I2C3_TX	I2C4_RX	-	-	I2C2_TX	-	I2C4_TX	-
Channel 9	-	SPI2_RX	-	-	-	-	SPI2_TX	-

RM0410 Table 27. DMA1 request mapping (p. 252). USART3_TX appears twice: Stream 3 / Channel 4 and Stream 4 / Channel 7.

Request	Stream	Channel
USART3_TX	DMA1 Stream 3	Channel 4 (used in this project)
USART3_TX	DMA1 Stream 4	Channel 7 (alternative)
USART3_RX	DMA1 Stream 1	Channel 4
USART2_TX (old F407 code)	DMA1 Stream 6	Channel 4

Caution from RM0410: the same peripheral request may be mapped to two channels, but the two must never be served at the same time, or the DMA behaves unpredictably.

In CMSIS a stream's registers are accessed as `DMA1_Stream3->CR`, `NDTR`, `PAR`, `M0AR`, `M1AR`, `FCR`. Stream register offset: 0x10 + 0x18 * x.

Remember: Stream = engine, channel = which peripheral may trigger it. Look up the pair in Table 27/28; USART3_TX is DMA1 S3 C4.

5. Arbiter

Each controller has one arbiter. It manages the 8 stream requests separately for each of the two AHB master ports (memory and peripheral), in two stages:

Stage	Rule
1. Software priority	PL[1:0] in DMA_SxCR (bits 17:16): 00 low, 01 medium, 10 high, 11 very high. Highest wins.
2. Hardware priority	If two streams have the same PL, the **lower stream number** wins (Stream 2 beats Stream 4).

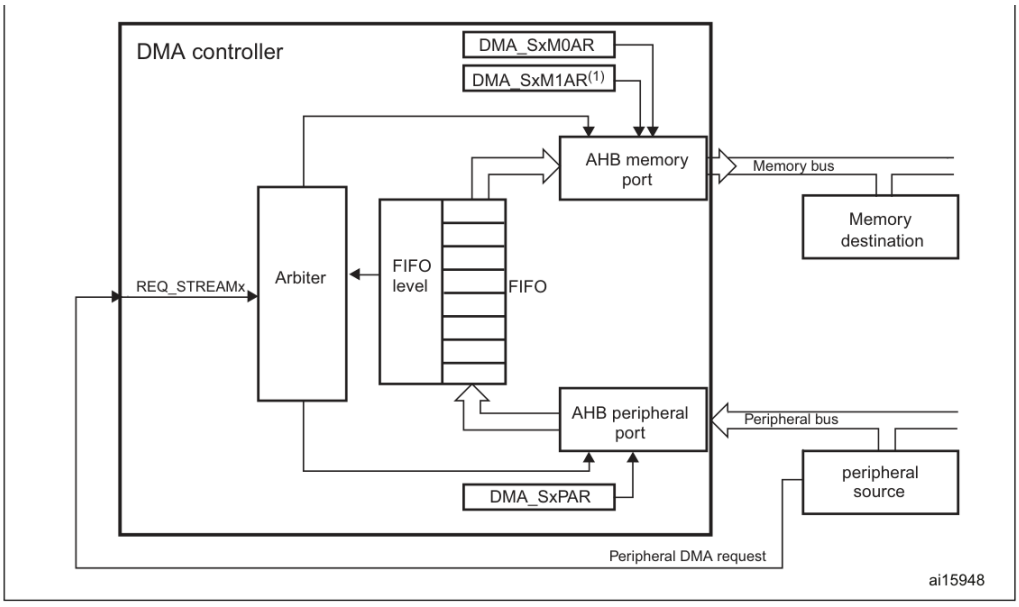
- DMA1 and DMA2 have separate arbiters. Conflicts between the two controllers, or with the CPU, are resolved by the bus matrix.
- In direct mode, RM0410 recommends a high priority for the stream to avoid FIFO overrun/underrun.
- This project uses one stream, so PL stays at reset (low).

Remember: Priority = PL first, then lower stream number. One arbiter per controller, per AHB port.

6. Types of transfer

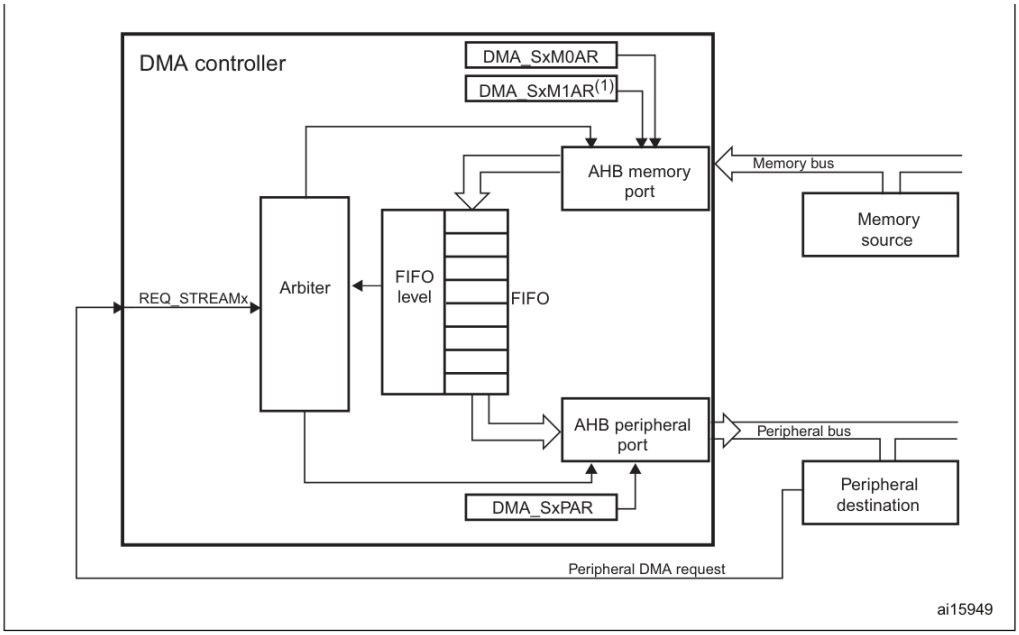
6.1 Direction: DIR[1:0] (bits 7:6)

DIR	Transfer	Source	Destination	Notes
00	Peripheral-to-memory (P2M)	PAR	M0AR	e.g. UART RX, ADC
01	Memory-to-peripheral (M2P)	M0AR	PAR	Used here: msg -> USART3->TDR
10	Memory-to-memory (M2M)	PAR	M0AR	DMA2 only, FIFO mode only, no circular



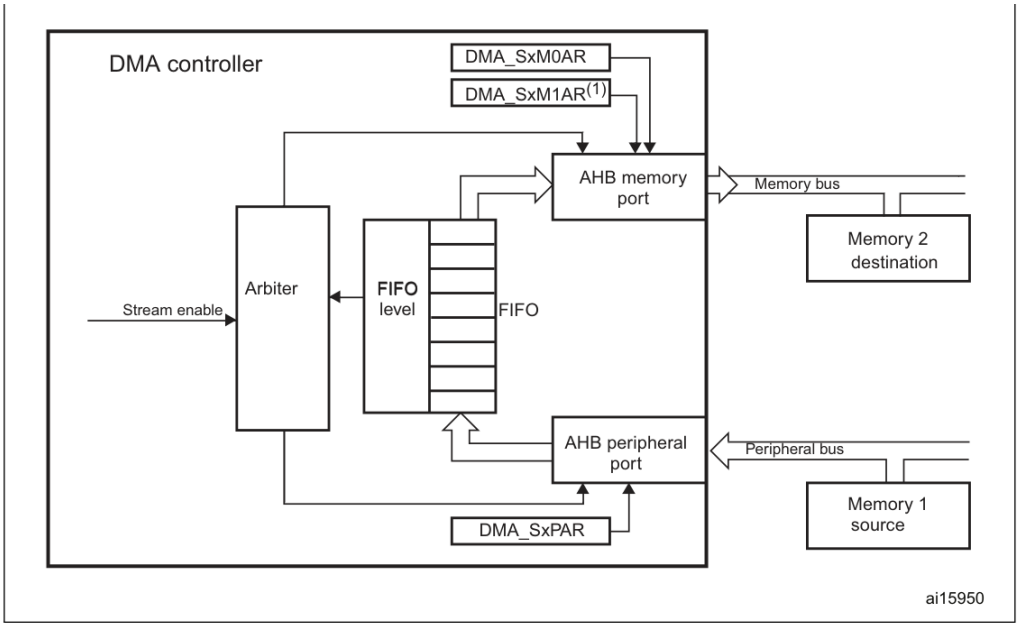
1. For double-buffer mode.

RM0410 Figure 25. Peripheral-to-memory mode (p. 255).



1. For double-buffer mode.

RM0410 Figure 26. Memory-to-peripheral mode (p. 256). This is the mode used in this project.



1. For double-buffer mode.

RM0410 Figure 27. Memory-to-memory mode (p. 257). Triggered by setting EN, not by a peripheral request.

6.2 Normal, circular and double-buffer modes

Mode	Bits	Behaviour
Normal	CIRC = 0	Stops when NDTR reaches 0; hardware clears EN. To send again: reload NDTR, clear flags, set EN. **Used here.**
Circular	CIRC = 1	NDTR and addresses reload automatically and requests keep being served. For continuous flows (ADC scan, UART RX ring buffer).
Double buffer	DBM = 1	Two memory pointers, M0AR and M1AR, swapped at each end of transaction. CT shows the current target. Circular is implied (CIRC ignored). Software may rewrite the *other* buffer address while the stream runs; writing the active one sets TEIF.

Circular mode with memory burst: NDTR must be a multiple of (Mburst beats x MSIZE / PSIZE), or data integrity is not guaranteed.

6.3 Flow controller: PFCTRL (bit 5)

- **0, DMA flow controller:** the length comes from NDTR (1 to 65535). Used here.
- **1, peripheral flow controller:** the peripheral signals the end of transfer. Only SDMMC supports this. Circular and double-buffer are forbidden in this case.

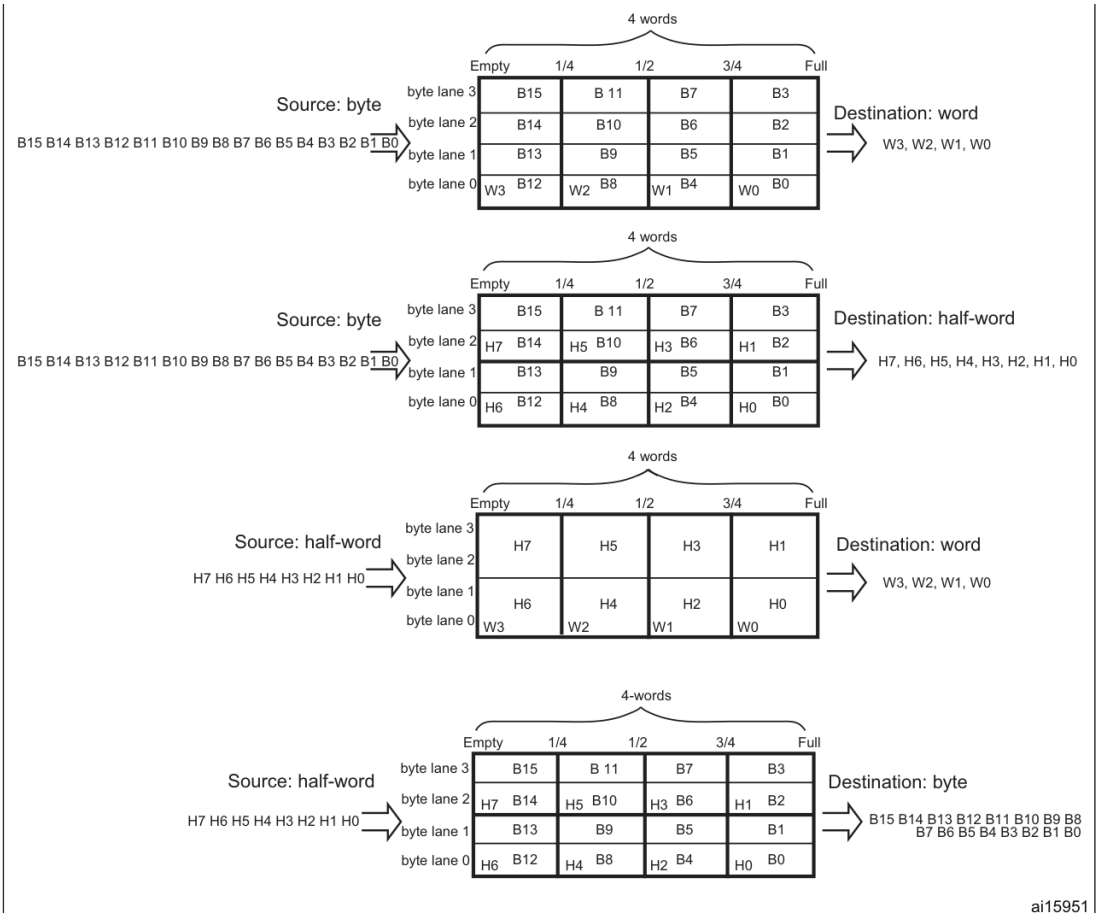
6.4 Data width and address increment

- PSIZE (12:11) and MSIZE (14:13): 00 byte, 01 half-word, 10 word. Here both are byte, because USART3->TDR takes one character per write.
- MINC (bit 10), PINC (bit 9): increment the memory or peripheral address after each item. For M2P to a USART, set MINC and leave PINC clear, so TDR stays fixed and the pointer walks the string.
- PINCOS (bit 15): when set, the peripheral address increments by 4 regardless of PSIZE.
- NDTR counts items of **PSIZE**, not bytes. If PSIZE < MSIZE, NDTR must be a multiple of MSIZE/PSIZE (RM0410 Table 32).

6.5 Direct mode vs FIFO mode: DMDIS (FCR bit 2)

	Direct mode (DMDIS = 0, reset)	FIFO mode (DMDIS = 1)
Buffering	Each request moves one item straight through. For M2P the DMA preloads one item so it is ready when the request comes.	Data goes through the 4-word FIFO.
Widths	MSIZE ignored, both sides use PSIZE.	PSIZE and MSIZE may differ (packing/unpacking).
Burst	Not possible.	Possible.
Memory-to-memory	Not allowed.	Required.
Threshold	Not used.	FTH[1:0]: 1/4, 1/2, 3/4 or full.

FCR resets to 0x21: direct mode, FTH = 1/2, FS = 100 (FIFO empty). This project sets FIFO mode with a full threshold; for a byte-for-byte transfer to a USART, direct mode would work equally well.



RM0410 Figure 28. FIFO structure (p. 262): how bytes, half-words and words are packed and unpacked in the 4-word FIFO.

6.6 Single vs burst transfer

- MBURST (24:23), PBURST (22:21): 00 single, 01 INCR4, 10 INCR8, 11 INCR16.
- Burst requires FIFO mode, and the memory burst must fit the FIFO threshold (Table 33). An incompatible combination sets FEIF when the stream is enabled.
- Single transfer is used here.

MSIZE	FIFO level	MBURST = INCR4	MBURST = INCR8	MBURST = INCR16
Byte	1/4	1 burst of 4 beats	Forbidden	Forbidden
	1/2	2 bursts of 4 beats	1 burst of 8 beats	
	3/4	3 bursts of 4 beats	Forbidden	
	Full	4 bursts of 4 beats	2 bursts of 8 beats	1 burst of 16 beats
Half-word	1/4	Forbidden	Forbidden	Forbidden
	1/2	1 burst of 4 beats		
	3/4	Forbidden		
	Full	2 bursts of 4 beats	1 burst of 8 beats	
Word	1/4	Forbidden	Forbidden	
	1/2			
	3/4			
	Full	1 burst of 4 beats		

RM0410 Table 33. FIFO threshold configurations (p. 263). Grey cells are forbidden.

6.7 Summary of allowed combinations

DMA transfer mode	Source	Destination	Flow controller	Circular mode	Transfer type	Direct mode	Double-buffer mode
Peripheral-to-memory	AHB peripheral port	AHB memory port	DMA	Possible	single	Possible	Possible
					burst	Forbidden	
			Peripheral	Forbidden	single	Possible	Forbidden
					burst	Forbidden	
Memory-to-peripheral	AHB memory port	AHB peripheral port	DMA	Possible	single	Possible	Possible
					burst	Forbidden	
			Peripheral	Forbidden	single	Possible	Forbidden
					burst	Forbidden	
Memory-to-memory	AHB peripheral port	AHB memory port	DMA only	Forbidden	single	Forbidden	Forbidden
					burst		

RM0410 Table 34. Possible DMA configurations (p. 267). Grey cells are forbidden.

Remember: M2P/P2M with DMA flow control can do everything. Peripheral flow control: no circular, no double buffer. M2M: DMA2 only, FIFO only, no circular, no double buffer. Burst never in direct mode.

7. Interrupts, flags and errors

Each stream has five events, all ORed into one NVIC line (for example `DMA1_Stream3_IRQn`). The handler must read the status register to see which event happened, and clear the flag, or the interrupt fires again forever.

Event	Enable bit	Flag	Stream 3 bit in LISR
Half transfer	CR.HTIE (3)	HTIF	26
Transfer complete	CR.TCIE (4)	TCIF	27
Transfer error	CR.TEIE (2)	TEIF	25
FIFO over/underrun	FCR.FEIE (7)	FEIF	22
Direct mode error	CR.DMEIE (1)	DMEIF	24

- Status: DMA_LISR (streams 0 to 3), DMA_HISR (streams 4 to 7). Read-only.
- Clear: write 1 to the same bit position in DMA_LIFCR / DMA_HIFCR (write-1-to-clear; reads as 0).
- Bit groups per stream: Stream 0 bits 0 to 5, Stream 1 bits 6 to 11, Stream 2 bits 16 to 21, Stream 3 bits 22 to 27. HISR repeats the pattern for streams 4 to 7.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF3	HTIF3	TEIF3	DMEIF 3	Res.	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF 2	Res.	FEIF2
				r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF1	HTIF1	TEIF1	DMEIF 1	Res.	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF 0	Res.	FEIF0
				r	r	r	r		r	r	r	r	r		r

RM0410 8.5.1 DMA_LISR bit layout (p. 270). DMA_LIFCR has the same layout with a C prefix (CTCIF3 ...).

Error causes (RM0410 8.3.19)

Flag	Set when
TEIF	Bus error on a DMA read or write, or writing the active memory address register in double-buffer mode. Hardware clears EN.
FEIF	FIFO underrun or overrun, or the stream is enabled with a FIFO threshold that does not fit the memory burst. In direct mode it can also mean the memory bus was not granted in time.
DMEIF	Only P2M, direct mode, MINC = 0: a new request came before the previous item reached memory, so two items went to the same address.

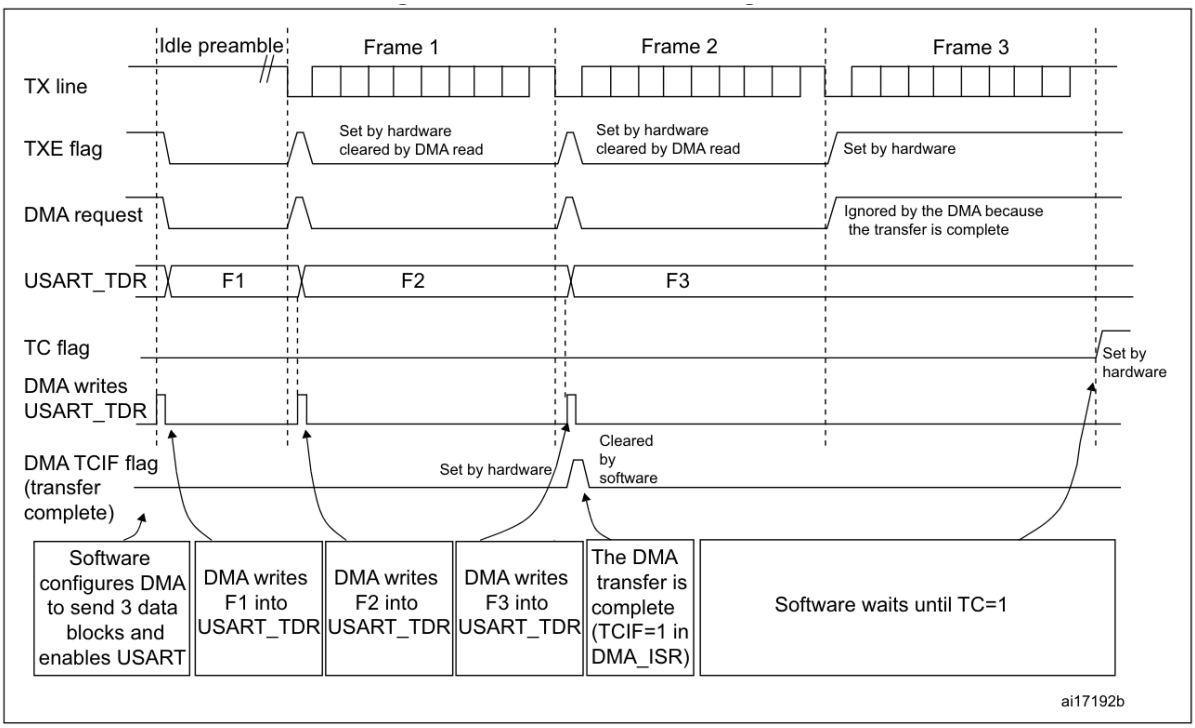
Remember: One IRQ per stream, five flags. Read LISR/HISR, clear with LIFCR/HIFCR using '='. Stream 3 = bits 22 to 27.

8. Enabling sequence

From RM0410 8.3.18 "Stream configuration procedure", with the USART side from 34.5.15 "USART continuous communication in DMA mode".

#	Step	For this project
1	Enable the DMA clock. Without it every DMA register write is ignored.	RCC->AHB1ENR |= RCC_AHB1ENR_DMA1EN
2	If EN is set, clear it and read back until 0. Configuration is only possible with EN = 0; the clear takes effect after the current transfer finishes.	CR &= ~DMA_SxCR_EN; while (CR & EN);
3	Clear all of the stream's flags from the previous transfer. They must be clear before the stream can be re-enabled.	DMA1->LIFCR = (0x3D << 22)
4	Peripheral address.	PAR = &USART3->TDR
5	Memory address (and M1AR for double buffer).	M0AR = msg
6	Number of items.	NDTR = sizeof(msg) - 1
7	Channel.	CHSEL = 4
8	Flow controller, if the peripheral controls it.	PFCTRL = 0
9	Priority.	PL = low
10	FIFO: direct or FIFO mode, threshold, FEIE.	FCR
11	Rest of CR: DIR, MINC/PINC, PSIZE/MSIZE, burst, CIRC/DBM, HTIE/TCIE/TEIE/DMEIE.	DIR = 01, MINC = 1
12	NVIC, if interrupts are used.	NVIC_EnableIRQ(DMA1_Stream3_IRQn)
13	Enable the stream. It can now serve requests.	CR |= DMA_SxCR_EN
14	Let the peripheral request DMA. TXE is already 1 when idle, so the first request is immediate.	USART3->CR3 |= USART_CR3_DMAT

Switching off: disable the DMA stream first and wait for EN = 0, and only then disable the peripheral (RM0410 warning in 8.3.18).



RM0410 Figure 394. Transmission using DMA (p. 1269). DMA TCIF sets when the last byte is written to TDR; USART TC sets later, when that byte has left the shift register.

Remember: Clock -> EN off and wait -> clear flags -> PAR, M0AR, NDTR -> CHSEL, PFCTRL, PL -> FCR -> CR -> NVIC -> EN on -> peripheral DMA request enable.

9. How this project uses DMA

Goal: send `"Hello World\r\n"` over USART3 (the ST-LINK virtual COM port) by DMA each time the user button B1 is used.

9.1 Configuration

Setting	Value
Controller / stream / channel	DMA1 / Stream 3 / Channel 4
Direction	M2P: msg (const, in Flash) -> USART3->TDR
NDTR	sizeof(msg) - 1 = 13 (the trailing NUL is not sent)
Widths	PSIZE = MSIZE = byte
Increment	MINC = 1, PINC = 0
Mode	Normal (not circular), DMA flow control
FIFO	FIFO mode, full threshold, single transfer
Priority	Low (reset value)
Interrupts	HT, TC, TE, DME, FE, via DMA1_Stream3_IRQHandler
USART3	PD8 TX / PD9 RX, AF7, BRR = 0x8B (115200 baud at 16 MHz HSI)

9.2 Startup order in main()

```
button_init();                    // PC13 -> EXTI13, falling edge
usart3_init();                    // PD8/PD9 AF7, 115200 baud
dma1_init();                      // clock + stream config (EN = 0)
dma1_interrupt_configuration();   // HT/TC/TE/DME/FE + NVIC
enable_dma1_stream3();            // EN = 1, stream waits for requests
```

After startup the stream is armed but idle, because USART3 is not yet allowed to request DMA (DMAT = 0).

9.3 Run-time flow

#	What happens	Where
1	Button edge -> EXTI13 -> sets DMAT in USART3->CR3.	EXTI15_10_IRQHandler
2	USART3 requests on each TXE; DMA1 S3 copies one byte msg[i] -> TDR per request.	Hardware
3	Half the bytes sent -> HTIF3.	HT_Complete_callback (empty)
4	Last byte written -> TCIF3; hardware clears EN.	DMA1_Stream3_IRQHandler
5	Re-arm: reload NDTR, clear DMAT, set EN. The stream waits again, as at startup.	FT_Complete_callback
6	TE / DME / FE -> callback stops in while (1) for inspection in the debugger.	*_error_callback

```
// stm32f7xx_it.c
void EXTI15_10_IRQHandler(void)
{
    USART3->CR3 |= (1 << 7);          // DMAT: let USART3 request DMA
    clear_exti_pending_bit();
}

// main.c
void FT_Complete_callback(void)
{
    DMA1_Stream3->NDTR = (uint16_t)sizeof(msg) - 1;
    USART3->CR3 &= ~(1 << 7);         // stop requests until the next press
    enable_dma1_stream3();
}
```

10. Things to watch

Pitfall	Why it matters
DMA clock not enabled first	All DMA register writes are silently ignored. Symptom: DMA1_Stream3 reads all zeros in the debugger.
Configuring with EN = 1	Writes are ignored or cause errors. Clear EN and wait for 0.
Stale flags at re-enable	All of the stream's flags must be clear before EN is set again. The current handler clears only the flag it handled, so HTIF can still be set when the stream is re-armed.
Empty or non-clearing IRQ handler	The flag stays set and the CPU re-enters the handler forever.
LIFCR/HIFCR with |=	They are write-1-to-clear and read as 0, so |= happens to work, but plain = is the correct idiom.
Wrong stream or missing CHSEL	The stream never sees the peripheral's requests. Check Table 27.
MINC not set	Every item is read from the same address (output: HHHH...).
Peripheral request not enabled	DMAT (USART), or the equivalent in other peripherals, is needed or no request reaches the DMA.
DMA TC is not USART TC	When TCIF3 sets, the last byte is still in the USART shift register. Wait for USART_ISR_TC before disabling the USART or entering low power.

Pitfall	Why it matters
Disabling the peripheral first	RM0410: disable the stream and wait for EN = 0 first.
Peripheral request enabled before the stream	AN4031 4.3: configure and enable the stream, then enable the peripheral's DMA request. The other way round can set FEIF in M2P, because the DMA is not ready with the first item. This project does it in the right order (DMAT is set on the button press).
Button edge	B1 on the Nucleo-144 is active-high, so a falling-edge trigger fires on release. The button is not debounced.
D-cache (F7)	If D-cache is on and the buffer is in SRAM: clean before M2P (SCB_CleanDCache_by_Addr), invalidate after P2M. Not an issue today: cache off, msg in Flash. See AN4839.

11. Register quick reference

DMA_SxCR, offset 0x10 + 0x18 * x, reset 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CHSEL[3:0]				MBURST[1:0]		PBURST[1:0]		Res.	CT	DBM	PL[1:0]	
			rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PINCOS	MSIZE[1:0]		PSIZE[1:0]		MINC	PINC	CIRC	DIR[1:0]		PFCTRL	TCIE	HTIE	TEIE	DMEIE	EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

RM0410 8.5.5 DMA_SxCR bit layout (p. 273).

Bits	Field	Meaning
0	EN	Stream enable. Cleared by hardware at end of transfer (normal mode) or on error.
1	DMEIE	Direct mode error interrupt enable
2	TEIE	Transfer error interrupt enable
3	HTIE	Half transfer interrupt enable
4	TCIE	Transfer complete interrupt enable
5	PFCTRL	Peripheral is the flow controller
7:6	DIR	00 P2M, 01 M2P, 10 M2M
8	CIRC	Circular mode
9	PINC	Peripheral address increment
10	MINC	Memory address increment
12:11	PSIZE	Peripheral data size: 00 byte, 01 half-word, 10 word
14:13	MSIZE	Memory data size
15	PINCOS	Peripheral increment offset fixed to 4
17:16	PL	Priority: low, medium, high, very high
18	DBM	Double-buffer mode
19	CT	Current target: 0 = M0AR, 1 = M1AR
22:21	PBURST	Peripheral burst: single, INCR4, INCR8, INCR16
24:23	MBURST	Memory burst
28:25	CHSEL	Channel select, 0 to 15

DMA_SxFCR, offset 0x24 + 0x18 * x, reset 0x0000 0021

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEIE	Res.	FS[2:0]			DMDIS	FTH[1:0]	
								rw		r	r	r	rw	rw	rw

RM0410 8.5.10 DMA_SxFCR bit layout (p. 278).

Bits	Field	Meaning
1:0	FTH	FIFO threshold: 00 1/4, 01 1/2 (reset), 10 3/4, 11 full
2	DMDIS	1 = FIFO mode, 0 = direct mode (reset)
5:3	FS	FIFO status, read-only: 000 <1/4 ... 100 empty, 101 full
7	FEIE	FIFO error interrupt enable

Register	Offset	Purpose
DMA_LISR / DMA_HISR	0x00 / 0x04	Status flags, streams 0-3 / 4-7
DMA_LIFCR / DMA_HIFCR	0x08 / 0x0C	Write 1 to clear the matching flag
DMA_SxNDTR	0x14 + 0x18 * x	Items left to transfer (1 to 65535)
DMA_SxPAR	0x18 + 0x18 * x	Peripheral address
DMA_SxM0AR	0x1C + 0x18 * x	Memory 0 address
DMA_SxM1AR	0x20 + 0x18 * x	Memory 1 address (double buffer)

12. Flashcards

Cover the right column, answer, then check. Revisit the ones you miss sooner.

Question	Answer
What is DMA, in one sentence?	A hardware bus master that copies data between peripherals and memory (or memory and memory) on request, without the CPU moving each item.
Name the three AHB ports of an STM32 DMA controller.	One slave port (CPU programs it) and two master ports: memory port and peripheral port.
Give four benefits of DMA according to AN4031.	Background transfers with CPU interrupted only per block; large data with little performance impact; central SRAM buffer cheaper in silicon/power; low latency via dual ports and direct APB paths.
How long does polling 13 bytes at 115200 8N1 keep the CPU busy, and how many cycles at 16 MHz?	13 x 10 / 115200 = about 1.13 ms, about 18,000 cycles.
Name five typical DMA use cases.	UART TX/RX, ADC sampling, SPI streaming, audio (SAI), DAC waveforms, timer-driven updates, SDMMC/QUADSPI, memory copy.
When is DMA not worth it?	Very short or rare transfers, where setup and interrupt cost exceed a polling loop.
What does AN4031 say about enable order between the stream and the peripheral?	Enable the stream first, then the peripheral's DMA request; otherwise FEIF may be set in M2P.
How many DMA controllers, streams per controller, and channels per stream does the F767 have?	2 controllers (DMA1, DMA2), 8 streams each, up to 16 channels per stream.
Which controller can do memory-to-memory, and why?	Only DMA2. DMA1's peripheral port is wired only to APB1, not to the bus matrix.
What is the difference between a stream and a channel?	A stream is the transfer engine. A channel is a peripheral request line; CHSEL picks which one triggers the stream.
Which stream and channel serve USART3_TX? Name both options.	DMA1 Stream 3 Channel 4, or DMA1 Stream 4 Channel 7.
Where are CHSEL's bits, and how wide is it on the F767?	DMA_SxCR bits 28:25, 4 bits (channels 0-15).
How does the arbiter decide between two streams?	First by PL (software priority), then the lower stream number wins on a tie.
What are the DIR encodings?	00 P2M, 01 M2P, 10 M2M.
In M2M mode, which register holds the source address?	DMA_SxPAR (destination is M0AR).
What does NDTR count?	Items of PSIZE, not bytes. Max 65535.
What happens at the end of a normal-mode transfer?	NDTR = 0, TCIF set, hardware clears EN. Re-arm by reloading NDTR, clearing flags and setting EN.
What does double-buffer mode do and which bit shows the active buffer?	Alternates M0AR / M1AR at each end of transaction; CT shows the current target. Circular is implied.
Name three restrictions of direct mode.	No burst, MSIZE ignored (PSIZE used for both), not allowed for M2M.
What is the FCR reset value and what does it mean?	0x21: direct mode, threshold 1/2, FIFO empty.
What are the five per-stream events?	Half transfer, transfer complete, transfer error, FIFO error, direct mode error.
Which bits in LISR belong to Stream 3?	Bits 22-27: FEIF 22, DMEIF 24, TEIF 25, HTIF 26, TCIF 27.
How do you clear a DMA flag?	Write 1 to the matching bit in LIFCR/HIFCR (use '=', not '|=').
What must be true before writing a stream's configuration registers?	EN must read 0.

Question	Answer
What must be true before setting EN again?	All of that stream's flags from the previous transfer must be cleared.
Which USART bit makes it request DMA for TX?	DMAT in USART_CR3 (bit 7).
When the DMA TCIF sets for a USART TX, is the data on the wire?	Not all of it: the last byte is still shifting out. Wait for USART TC.
Correct order to switch off a peripheral used by DMA?	Disable the DMA stream, wait for EN = 0, then disable the peripheral.
When is DMEIF set?	Only P2M, direct mode, MINC = 0, when a request arrives before the previous item reached memory.
Why might a burst configuration fail at enable?	The FIFO threshold doesn't fit the memory burst (Table 33): FEIF is set.
Why do DMA and CPU see different data on an F7, and the fix?	D-cache. Clean the cache before M2P, invalidate after P2M (or use a non-cacheable region).
Symptom: DMA registers read all zero after init. Most likely cause?	The DMA clock (RCC_AHB1ENR_DMA1EN) was never enabled.

13. References

Document	Used for
ST RM0410 Rev 5, STM32F76xxx and STM32F77xxx advanced Arm-based 32-bit MCUs reference manual	2.1 system architecture (Fig 1); 4.3 low-power modes; 5.3.15 RCC_AHB1LPENR; chapter 8 DMA (8.1-8.5, Figs 23-28, Tables 27-35, register descriptions); 34.5.15 USART DMA (Fig 394)
ST AN4031 Rev 3 (DocID022648), Using the STM32F2, STM32F4 and STM32F7 Series DMA controller, June 2016	1 DMA description and benefits; 2.1.3 bus arbitration delay; 2.2.1 dual DMA port paths; 3.2 ADC and SPI examples; 4.1-4.3 disable/flag/enable sequences; 4.10 F7 cache maintenance; 5 conclusion
ST UM1974, STM32 Nucleo-144 boards user manual	6.9 USART communication: USART3 on PD8/PD9 to the ST-LINK virtual COM port (SB5/SB6 on)
This project: app/src/main.c, drivers/src/stm32f7xx_it.c	Section 9 configuration and run-time flow

All documents are ST's public technical literature, available from st.com; local copies are in ~/Documents/Projects/STM32_docs/STM32F7/.